

Fonctions

Fonction : Définition

- Une fonction permet de **décomposer un programme complexe en une série de sous-programmes plus simples**, lesquels peuvent à leur tour être décomposés en fragments plus petits etc.
- Une fonction est **réutilisable**
- Une fonction apparaît sous la forme d'**un nom associé à des parenthèses** :

`print()`

- **Dans les parenthèses**, on transmet à la fonction **un ou plusieurs arguments**
- La fonction **fournit une valeur de retour**

Fonctions prédéfinies

`print()`

`input ()`

Voir le document sur ENT (liste de fonctions)

- `print()`:
 - affiche des données
- `type()` :
 - affiche le type d'une donnée Python
- `len()` :
 - renvoie la longueur ou le nombre de valeurs d'une donnée de type séquence ou collection
- `input()`
 - permet de dialoguer et d'échanger des données avec l'utilisateur
 - on passe un message en argument de la fonction

```
[>>> prenom = input("Rentrez votre prénom : ")
Rentrez votre prénom : Pierre
[>>> prenom
'Pierre']
```

Fonctions de conversion

- `str()`

- retourne une chaîne de caractères à partir d'une donnée qu'on va lui passer en argument ;

- `int()`

- retourne un entier à partir d'un nombre ou d'une chaîne contenant un nombre qu'on va lui passer en argument ;

- `float()`

- retourne un nombre décimal à partir d'un nombre ou d'une chaîne contenant un nombre qu'on va lui passer en argument ;

- `complex()`

- retourne un nombre complexe à partir d'un nombre ou d'une chaîne contenant un nombre qu'on va lui passer en argument ;

Fonctions de conversion

- `bool()`
 - retourne un booléen à partir d'une donnée qu'on va lui passer en argument ;
- `list()`
 - retourne une liste à partir d'une donnée itérable (une donnée dont on peut parcourir les valeurs) ;
- `tuple()`
 - retourne un tuple à partir d'une donnée itérable ;
- `dict()`
 - crée un dictionnaire à partir d'un ensemble de paires clef = "valeur" ;
- `set()`
 - retourne un ensemble (set) à partir d'une donnée itérable.

```
[>>> str(29)
'29'
[>>> int("29")
29
[>>> float(29)
29.0
[>>> float("29.3")
29.3
[>>> bool("Pierre")
True
[>>> bool("")
False
[>>> bool("false")
True
[>>> bool("False")
True
[>>> bool(False)
False
[>>> list("Pierre")
['P', 'i', 'e', 'r', 'r', 'e']
[>>> tuple("Pierre")
('P', 'i', 'e', 'r', 'r', 'e')
[>>> set("Pierre")
{'P', 'e', 'i', 'r'}
```

Fonctions mathématiques

- `range()`
 - renvoie une séquence de nombres. On peut lui passer jusqu'à 3 arguments mais 1 seul est obligatoire.
- `round()`
 - permet d'arrondir un nombre spécifié en argument `t` à l'entier le plus proche avec un degré de précision (un nombre de décimales) éventuellement spécifié en deuxième argument.
 - le nombre de décimales par défaut est 0, ce qui signifie que la fonction retournera l'entier le plus proche.

```
[>>> round(-2.7)
-3
[>>> round(1.2)
1
[>>> round(1.5)
2
[>>> round(1.6)
2
```


Fonctions mathématiques

- **sum()**
 - permet de calculer une somme.
 - on peut lui passer une liste de nombres en arguments.
 - on peut également lui passer une valeur “de départ” en deuxième argument qui sera ajoutée à la somme calculée.
- **max()**
 - retourne la plus grande valeur d’une donnée itérable, cad d’une donnée dont on peut parcourir les différentes valeurs
 - on peut lui passer autant d’arguments qu’on veut
- **min()**
 - retourne la plus petite valeur d’une donnée itérable

```
x=[1,2]
(sum(x,2))
=> 5
sum([1,2])
=> 3
```

```
[>>> x = [-8, 32, 4, 172]
[>>> sum(x)
200
[>>>
[>>> max(x)
172
[>>> min(x)
-8
```

Fonctions originales

Créer une nouvelle fonction permet de

- **Donner un nom à tout un ensemble d'instructions**
- **Simplifier le corps principal d'un programme**
- **Raccourcir un programme** par élimination des portions de code qui se répètent
- **Ne pas réécrire** à chaque fois l'algorithme qui accomplit ce travail

Définir une fonction

```
def nomDeLaFonction(liste de paramètres):  
    ...  
    bloc d'instructions  
    ...
```

- Nom
 - N'importe quel nom sauf des mots réservés du langage, aucun caractère spécial (sauf _)
 - Conseillée : tout en minuscules (maj. Réservé aux classes)
 - Nom très **explicite**
- Paramètres
 - la **liste de paramètres** spécifie quelles informations il faut fournir en **guise d'arguments**
 - Les parenthèses restent vides si la fonction ne nécessite pas d'arguments
- Appel de fonction
 - **Nom de la fonction suivi de parenthèses** vides ou avec les arguments

Fonction simple sans paramètres

```
>>> def table7():  
...     n = 1  
...     while n <11 :  
...         print(n * 7, end = ' ')  
...         n = n +1  
... 
```

- Fonction qui calcule et affiche les 10 premiers termes de la table de multiplication par 7
- Pour utiliser la fonction

```
>>> table7()
```

provoque l'affichage de :

```
7 14 21 28 35 42 49 56 63 70
```

Répéter la réutilisation de la fonction

- Réutiliser la fonction à plusieurs reprises
- Incorporer cette fonction dans la définition d'une autre

```
>>> def table7triple():  
...     print('La table par 7 en triple exemplaire :')  
...     table7()  
...     table7()  
...     table7()  
...
```

- Utiliser cette nouvelle fonction

```
>>> table7triple()
```

l'affichage résultant devrait être :

```
La table par 7 en triple exemplaire :  
7 14 21 28 35 42 49 56 63 70  
7 14 21 28 35 42 49 56 63 70  
7 14 21 28 35 42 49 56 63 70
```

Fonction avec paramètre

- Si l'on veut afficher maintenant une table de multiplication par 9 et après par 13
 - ⇒ Définir **une fonction qui affiche n'importe quelle table de multiplication à la demande**
 - ⇒ Si l'on appelle cette nouvelle fonction, on doit indiquer quelle table on veut afficher
 - ⇒ Cette information s'appelle un **argument**
 - ⇒ Il faut prévoir une variable particulière pour recevoir l'argument transmis, elle s'appelle un **paramètre**

```
>>> def table(base):  
...     n = 1  
...     while n < 11 :  
...         print(n * base, end = ' ')  
...         n = n + 1
```

print () #pour avoir un saut de ligne à l'affichage

- Pour tester cette nouvelle fonction, on l'appelle avec un argument

```
>>> table(13)  
13 26 39 52 65 78 91 104 117 130
```

```
>>> table(9)  
9 18 27 36 45 54 63 72 81 90
```

Si vous avez « none » nous n'avons pas renvoyé une valeur d'une fonction, car la fonction fournit une valeur de retour. Ici ce n'est pas une vraie fonction. (voir plus tard)

Utilisation d'une variable comme argument

- L'argument de la fonction précédente est toujours une constante (9, 13, etc.), mais il peut être une variable

```
>>> a = 1
>>> while a < 20:
...     table(a)
...     a = a + 1
... 
```

- Le nom d'une variable qu'on passe comme argument n'a rien à voir avec le nom du paramètre correspondant dans la fonction

Fonction avec plusieurs paramètres

- La fonction **table()** n'affiche que les 10 premiers termes de la table de multiplication, mais on veut souhaiter qu'elle en affiche d'autres

```
>>> def tableMulti(base, debut, fin):  
...     print('Fragment de la table de multiplication par', base, ':')  
...     n = debut  
...     while n <= fin :  
...         print(n, 'x', base, '=', n * base)  
...         n = n + 1
```

- Elle utilise 3 paramètres : base de la table, indice du premier et du dernier terme à afficher

```
>>> tableMulti(8, 13, 17)
```

ce qui devrait provoquer l'affichage de :

```
Fragment de la table de multiplication par 8 :  
13 x 8 = 104  
14 x 8 = 112  
15 x 8 = 120  
16 x 8 = 128  
17 x 8 = 136
```

Variables locales / globales

- Variables locales
 - Ne sont accessibles qu'à la fonction elle-même
 - Le cas des variables *base*, *debut*, *fin*
 - Chaque fois que la fonction *tableMulti()* est appelé, **Python réserve pour elle un nouvel espace de noms qui est inaccessible depuis l'extérieur de la fonction**

```
>>> print(base)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'base' is not defined
```

Variables locales / globales

- Variables globales
 - Sont **définies à l'extérieur d'une fonction**
 - Leur contenu est « visible » de l'intérieur, mais la fonction ne peut pas le modifier

```
>>> def mask():  
...     p = 20  
...     print(p, q)  
...  
>>> p, q = 15, 38  
>>> mask()  
20 38  
>>> print(p, q)  
15 38
```

- Variable p est locale car elle est définie à l'intérieur de la fonction
 - p et q sont des variables globales
- => Variable p est utilisé à deux reprises pour définir deux variables différentes

Variables locales / globales

- Il est utile que des variables soient définies comme locales car
 - on pourra toujours utiliser une infinité de fonctions sans se préoccuper des noms de variables qui y sont utilisées
 - ces **variables ne pourront jamais interférer avec celles qu'on aurait défini par ailleurs**

Modifier une variable globale

- L'instruction **global** : permet d'indiquer à l'intérieur de la définition d'une fonction quelles sont les variables à traiter globalement

```
>>> def monter():  
...     global a  
...     a = a+1  
...     print(a)  
...  
>>> a = 15  
>>> monter()  
16  
>>> monter()  
17  
>>>
```

- Variable *a* est accessible et modifiable car elle est signalée explicitement étant une variable à traiter globalement

Vraies fonctions

- Une vraie fonction doit renvoyer une valeur
- Modifier la fonction **table ()** pour qu'elle renvoie une valeur

```
>>> def table(base):  
...     resultat = []           # resultat est d'abord une liste vide  
...     n = 1  
...     while n < 11:  
...         b = n * base  
...         resultat.append(b)  # ajout d'un terme à la liste  
...         n = n + 1           # (voir explications ci-dessous)  
...     return resultat  
...
```

- L'instruction **return** définit ce que doit être la valeur « renvoyée » par la fonction : ici le contenu de la variable *resultat* = la liste des nombres générés
- L'instruction **resultat.append(b)** : application de la méthode **append()** à l'objet **resultat** qui est une liste
- Une méthode est une fonction qui est associée à un objet
- Sous Python, les listes constituent une classe particulière d'objets

Vraies fonctions

```
>>> def table(base):  
...     resultat = []           # resultat est d'abord une liste vide  
...     n = 1  
...     while n < 11:  
...         b = n * base  
...         resultat.append(b)  # ajout d'un terme à la liste  
...         n = n + 1           # (voir explications ci-dessous)  
...     return resultat  
...
```

- Tester cette fonction

```
>>> table(9)
```

ou

```
>>> ta9 = table(9)  
>>> print(ta9)  
[9, 18, 27, 36, 45, 54, 63, 72, 81, 90]  
>>> print(ta9[0])  
9  
>>> print(ta9[3])  
36  
>>> print(ta9[2:5])  
[27, 36, 45]  
>>>
```


Utilisation des fonctions dans un script

```
def cube(n):  
    return n**3  
  
def volumeSphere(r):  
    return 4 * 3.1416 * cube(r) / 3  
  
r = input('Entrez la valeur du rayon : ')  
print('Le volume de cette sphère vaut', volumeSphere(float(r)))
```

- Le programme qui calcule le volume d'une sphère à l'aide de la formule : $V = \frac{4}{3} \pi R^3$
- Le programme comporte 3 parties : les deux fonctions et le corps principal du programme
- Dans ce corps, on appelle la fonction **volumeSphere()** en lui transmettant la valeur entrée par l'utilisateur pour le rayon, préalablement convertie en un nombre réel à l'aide de la fonction intégrée **float()**

Utilisation des fonctions dans un script

- **Dans un script, la définition des fonctions doit précéder leur utilisation**

Intervertissez cet ordre (en plaçant par exemple le corps principal du programme au début).

Utilisation des commentaires dans une fonction

- Une chaine de car. ne joue aucun rôle fonctionnel dans le script : elle est **traitée par Python comme un simple commentaire**
- Python **place cette chaine dans une variable spéciale dont le nom est `__doc__`** et qui est associée à l'objet fonction comme étant l'un de ses attributs

```
>>> def essai():
...     "Cette fonction est bien documentée mais ne fait presque rien."
...     print("rien à signaler")
...
>>> essai()
rien à signaler
>>> print(essai.__doc__)
Cette fonction est bien documentée mais ne fait presque rien.
```

Typage des paramètres

- Le typage des variables est un typage dynamique : **le type d'une variable est défini au moment où on lui affecte une valeur, idem pour les paramètres d'une fonction**

```
>>> def afficher3fois(arg):  
...     print(arg, arg, arg)  
...  
  
>>> afficher3fois(5)  
5 5 5  
  
>>> afficher3fois('zut')  
zut zut zut  
  
>>> afficher3fois([5, 7])  
[5, 7] [5, 7] [5, 7]  
  
>>> afficher3fois(6**2)  
36 36 36
```

- La même fonction `afficher3fois()` accepte dans tous les cas l'argument qu'on lui transmet

Valeurs par défaut pour les paramètres

- Définir un argument par défaut pour chacun des paramètres permet d'obtenir **une fonction qui peut être appelée avec une partie seulement des arguments attendus**

```
>>> def politesse(nom, vedette = 'Monsieur'):  
...     print("Veuillez agréer ", vedette, nom, ", mes salutations cordiales.")  
...  
  
>>> politesse('Dupont')  
Veuillez agréer , Monsieur Dupont , mes salutations cordiales.  
  
>>> politesse('Durand', 'Mademoiselle')  
Veuillez agréer , Mademoiselle Durand , mes salutations cordiales.
```

- On peut définir une valeur par défaut pour tous les paramètres, ou une partie d'entre eux seulement.
- Dans ce cas, les paramètres sans valeur par défaut doivent précéder les autres dans la liste.
- Cette définition est incorrecte :

```
>>> def politesse(vedette ='Monsieur', nom):
```

Arguments avec étiquettes

- Les arguments que l'on fournit lors de l'appel d'une fonction **doivent être fournis exactement dans le même ordre que celui des paramètres** qui leur correspondent dans la définition de la fonction
- Si les paramètres annoncés dans la définition de la fonction **ont reçu chacun une valeur par défaut**, on peut faire appel à la fonction en fournissant les arguments correspondants dans **n'importe quel ordre**

```
>>> def oiseau(voltage=100, etat='allumé', action='danser la java'):  
...     print('Ce perroquet ne pourra pas', action)  
...     print('si vous le branchez sur', voltage, 'volts !')  
...     print("L'auteur de ceci est complètement", etat)  
...
```

```
>>> oiseau(etat='givré', voltage=250, action='vous approuver')  
Ce perroquet ne pourra pas vous approuver  
si vous le branchez sur 250 volts !  
L'auteur de ceci est complètement givré
```

```
>>> oiseau()  
Ce perroquet ne pourra pas danser la java  
si vous le branchez sur 100 volts !  
L'auteur de ceci est complètement allumé
```

une fonction qui renvoie un tuple

```
def test():  
    return 3, 4
```

```
>>> a = test()  
>>> a  
(3, 4)  
>>> b, c = test()  
>>> b  
3  
>>> c  
4
```

Note

Après **return**, il aurait été possible d'utiliser une notation avec des parenthèses pour le tuple, par exemple **return (3, 4)**.

- Fonction renvoyant plusieurs valeurs => utilisation de tuples

```
1 def decomposer(entier, divise_par):  
2     """Cette fonction retourne la partie entière et le reste de  
3     entier / divise_par"""  
4  
5     p_e = entier // divise_par  
6     reste = entier % divise_par  
7     return p_e, reste
```

Et on peut ensuite capturer la partie entière et le reste dans deux variables, au retour de la fonction :

```
1 >>> partie_entiere, reste = decomposer(20, 3)  
2 >>> partie_entiere  
3 6  
4 >>> reste  
5 2  
6 >>>
```

Là encore, on passe par des tuples sans que ce soit indiqué explicitement à Python. Si vous essayez de faire `retour = decomposer(20, 3)`, vous allez capturer un tuple contenant deux éléments : la partie entière et le reste de 20 divisé par 3.

Signature d'une fonction

- La signature d'une fonction en Python est son nom
- On ne peut donc pas définir deux fonctions du même nom (dans le cas inverse : l'ancienne définition est écrasée par la nouvelle)

Contrôle du flux d'exécution à l'aide d'un dictionnaire

Diriger l'exécution d'un programme dans différentes directions, en fonction de la valeur prise par une variable.

```
matériau = input("Choisissez le matériau : ")

dico = {'fer':fonctionA,
        'bois':fonctionC,
        'cuivre':fonctionB,
        'pierre':fonctionD,
        ... etc ...}

dico[matériau]()
```

- on définit un dictionnaire **dico** dans lequel les clés sont les différentes possibilités pour la variable **matériau**, et les valeurs, les noms des fonctions à invoquer en correspondance
- on peut améliorer la technique en remplaçant l'instruction **dico[matériau]()** par sa variante qui fait appel à la méthode **get()** afin de prévoir le cas où la clé demandée n'existerait pas dans le dictionnaire

```
dico.get(matériau, fonctAutre)()
```

- lorsque la valeur de la variable **matériau** ne correspond à aucune clé du dictionnaire, c'est la fonction **fonctAutre()** qui est invoquée.

Module de fonctions

- Module : **un bout de code** que l'on a **enfermé dans un fichier**
- Il est conseillé de placer fréquemment **les définitions de fonctions dans un module Python**, et le programme qui les utilise dans un autre.
- On peut l'utiliser dans n'importe quel autre script
- Si l'on veut travailler avec les fonctionnalités prévues par le module et utiliser ensuite toutes les fonctions et variables prévues, on **importe le module**

Importer un module de fonctions

- Les **modules** sont des fichiers qui regroupent des ensembles de fonctions
- Module *math* : fonctions mathématiques *sinus*, *cosinus*, *racine carrée*, etc.

```
>>> import math
```

```
>>> math.sqrt(16)
```

```
4
```

Module de fonction

- Aide pour avoir des informations sur le module :

```
>>> help("math")
```

```
>>> help("math.sqrt")
```

- En tapant **import math**, un espace de noms dénommé « **math** » , contenant les variables et fonctions du module « **math** » se crée.
- En tapant **math.sqrt(25)**, on précise à Python que l'on souhaite exécuter la fonction **sqrt** contenue dans l'espace de noms **math**
- On peut donc avoir **une autre fonction sqrt**, p.ex. **qu'on crée nous-mêmes** sans conflit avec celle du module **math** (**pas le même espace de noms**)

Importer un module de fonctions

- Autre méthode d'importation

```
from math import ...
```

Utile si l'on veut une/des fonctions particulières du module :

```
>>>from math import fabs    #fonction renvoyant la valeur absolue d'une var.
```

```
>>>fabs(-5)
```

```
5
```

```
>>>fabs(2)
```

```
2
```

Si l'on veut toutes les fonctions :

```
from math import *
```

S'amuser un peu : module *turtle*

- Ecrire des scripts qui réalisent des dessins suivant un modèle imposé à l'avance.
- Les principales fonctions :

<code>reset()</code>	On efface tout et on recommence
<code>goto(x, y)</code>	Aller à l'endroit de coordonnées <code>x, y</code>
<code>forward(distance)</code>	Avancer d'une distance donnée
<code>backward(distance)</code>	Reculer
<code>up()</code>	Relever le crayon (pour pouvoir avancer sans dessiner)
<code>down()</code>	Abaissier le crayon (pour recommencer à dessiner)
<code>color(couleur)</code>	<i>couleur</i> peut être une chaîne prédéfinie ('red', 'blue', etc.)
<code>left(angle)</code>	Tourner à gauche d'un angle donné (exprimé en degrés)
<code>right(angle)</code>	Tourner à droite
<code>width(épaisseur)</code>	Choisir l'épaisseur du tracé
<code>fill(1)</code>	Remplir un contour fermé à l'aide de la couleur sélectionnée
<code>write(texte)</code>	<i>texte</i> doit être une chaîne de caractères

- Essayez :

```
>>> from turtle import *  
>>> forward(120)  
>>> left(90)  
>>> color('red')  
>>> forward(80)
```

L'exercice est évidemment plus riche si l'on utilise des boucles :

```
>>> reset()  
>>> a = 0  
>>> while a < 12:  
    a = a + 1  
    forward(150)  
    left(150)
```

Travail à faire

- Définissez une fonction qui renvoie le nom du n-ième mois de l'année. Exemple, l'exécution de l'instruction :

`print(fonction(4))` doit donner le résultat : **Avril**.

- Faire une fonction qui affiche la table de multiplication par nombre (nb) en indiquant que le nombre maximum d'affichage doit être 10 par défaut

Travail à faire

- Définissez une fonction **changeCar(ch,ca1,ca2,debut,fin)** qui remplace tous les caractères **ca1** par des caractères **ca2** dans la chaîne de car. **ch**, à partir de l'indice **debut** et jusqu'à l'indice **fin**, ces deux derniers arguments pouvant être omis (et dans ce cas la chaîne est traitée d'une extrémité à l'autre) :

```
>>> phrase = 'Ceci est une toute petite phrase.'
```

```
>>> print(changeCar(phrase, ' ', '*'))
```

```
Ceci*est*une*toute*petite*phrase.
```

```
>>> print(changeCar(phrase, ' ', '*', 8, 12))
```

```
Ceci est*une*toute petite phrase.
```

```
>>> print(changeCar(phrase, ' ', '*', 12))
```

```
Ceci est une*toute*petite*phrase.
```

```
>>> print(changeCar(phrase, ' ', '*', fin = 12))
```

```
Ceci*est*une*toute petite phrase.
```

- Définir une fonction qui renvoie l'index de l'élément ayant la valeur la plus élevée dans la liste transmise en argument :

```
serie = [5, 8, 2, 1, 9, 3, 6, 7]
```

```
print(fonction(serie))
```

```
4
```

- Définir une fonction qui renvoie l'élément ayant la plus grande valeur dans la liste transmise. Les deux arguments **debut** et **fin** indiqueront les indices entre lesquels doit s'exercer la recherche, et chacun d'eux pourra être omis.

Exemples de la fonctionnalité attendue :

```
>>> serie = [9, 3, 6, 1, 7, 5, 4, 8, 2]
```

```
>>> print(fonction(serie))
```

```
9
```

```
>>> print(fonction(serie, 2, 5))
```

```
7
```

```
>>> print(fonction(serie, 2))
```

```
8
```

```
>>> print(fonction(serie, fin =3, debut =1))
```

```
6
```

Travail à faire

- Faire une fonction qui construit un dictionnaire inversé pour lequel les valeurs seront les clés et réciproquement.

Travail à faire

Ecrire un script qui crée et consulte le dictionnaire avec les informations sur les noms, l'âge et la taille des gens.

Astuces :

- Le script devra comporter deux fonctions : le remplissage du dictionnaire, et sa consultation.
- Remplir le dico en utilisant une boucle pour accepter les données entrées par l'utilisateur.
- Dans le dictionnaire, le nom servira de clé d'accès, et les valeurs seront constituées de tuples (âge, taille)
- L'âge (une année) est une donnée de type entier, et la taille est une donnée de type réel.
- Consulter le dico en utilisant aussi une boucle, dans laquelle l'utilisateur pourra fournir un nom quelconque pour obtenir en retour le couple « âge, taille » correspondant.
- Le résultat de la requête devra avoir une forme suivante :
« Nom : Iris Taravella - âge : 18 ans - taille : 1.55 m ».